

May 2026 - 7

The Cluster-Aware Smart Driver, Measured



Six load_balance settings, six real connection-distribution outcomes -- measured, not assumed

Figure 1: Figure 0: The Cluster-Aware Smart Driver, Measured

Welcome to this edition of yugabyteDB Developer’s Notebook (YDN). This month we answer the following question(s);

Our application connects to yugabyteDB through one hostname today – a load balancer in front of one region. We keep hearing yugabyteDB has a “smart driver” that’s cluster-aware and does its own load balancing. What does that actually mean in practice, and can we see it really distributing connections rather than just taking that on faith ?

A fair thing to want measured rather than assumed, and this month’s article does exactly that – six real connection-distribution outcomes, one per `load_balance` driver setting, captured directly from a live 6-node cluster (3 primary, 3 read-replica). We also cover exactly how the driver discovers cluster topology in the first place, and a real, practical detail that trips people up: how the driver reacts differently to two different kinds of node blacklisting.

Software versions The primary software components used in this edition of YDN include yugabyteDB Anywhere (YBA) and yugabyteDB (YB) version 2026.1.1.0, and the yugabyteDB Smart Driver (JDBC/psycopg-family cluster-aware driver). All of the steps below are run on one very large sized Mac Book Pro, or if you prefer, run these steps on yugabyteDB Aeon, yugabyteDB’s managed service, or Amazon Web Services (AWS), Google Cloud Platform (GCP), or another hyper-scaler.

For isolation and (simplicity), we develop and test all systems inside virtual machines or emulated machines, using the desktop hypervisors VMware Fusion version 13.6.4, and/or UTM version 4.6.5. Generally we run a single node of YBA, and 8 nodes of YB (5 nodes in one region, and 3 nodes in a second region) using Alma Linux versions 8 and 9, and Ubuntu Desktop version 24.04. We also run a client node using Ubuntu Desktop version 24.04.

7.1 Terms and core concepts

7.1.1 Introduction: How the Driver Learns About the Cluster

A yugabyteDB Smart Driver doesn’t rely solely on whatever `host/port` an application supplies at connection time – those values are only the *initial contact point*, used to discover the rest of the cluster, not the final set of connections the driver will actually use. Internally, the driver calls the same information exposed by `SELECT`

* FROM `yb_servers()`; – a live, queryable list of every tserver, its node type (`primary` or `read_replica`), and its cloud/region/zone placement:

Example 7-1: What `yb_servers()` actually returns

host	port	node_type	cloud	region	zone
D1-Yuga-C6N1	5433	primary	onprem	my_region1	my_zone1
D2-Yuga-C6N2	5433	primary	onprem	my_region1	my_zone1
D3-Yuga-C6N3	5433	primary	onprem	my_region1	my_zone1
D4-Yuga-C6N4	5433	read_replica	onprem	my_region2	my_zone4
D5-Yuga-C6N5	5433	read_replica	onprem	my_region2	my_zone4
D6-Yuga-C6N6	5433	read_replica	onprem	my_region2	my_zone4

[!NOTE] Because the `host` value supplied at connection time is only a starting point, not the final connection set, it should genuinely include nodes from more than one zone – if the single contact-point node happens to be down at connection time, a driver with only that one address has nothing to discover topology from at all. This is a resilience detail easy to overlook when a demo or a quick test only ever points at one convenient node.

7.1.2 Six Settings, Six Measured Outcomes

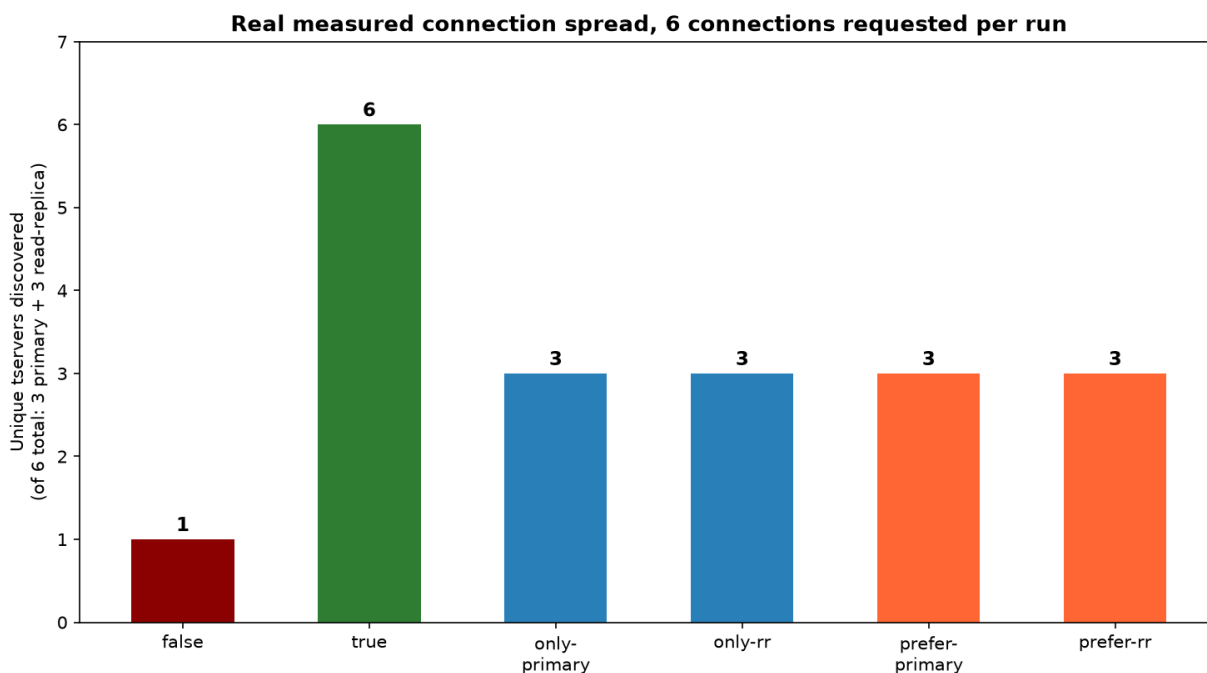


Figure 2: Figure 1: Real measured connection spread

The driver's `load_balance` setting controls whether – and how – it spreads connections across the discovered topology. Rather than take the documented behavior on faith, this project ran the identical test (acquire 6 connections, then report which physical tserver each one actually landed on) once per setting, against the real 6-node cluster above:

load_balance setting	Unique tservers used (of 6 total)	Real, measured behavior
false (default)	1	All 6 connections stayed on the single contact-point node – no distribution at all.
true / any	6	Connections spread perfectly evenly, one per node, across every primary and read-replica.
only-primary	3	Connections stayed exclusively within the 3 primary nodes – read-replicas never used.
only-rr	3	Connections stayed exclusively within the 3 read-replica nodes – primaries never used.
prefer-primary	3	Connections stayed within the 3 primaries – confirming “prefer” means “stay within this set while it’s available,” not “mix, but lean toward.”
prefer-rr	3	Symmetric result – stayed within the 3 read-replicas while they remained available.

The `false` result is worth dwelling on, since it’s the default: an application that never explicitly sets `load_balance` gets **zero** connection distribution – every connection lands on whichever single node it initially connected to, regardless of how many other nodes exist in the cluster. Smart-driver load balancing is real and measured here, but it is not automatic; it has to be explicitly requested.

[!NOTE] The `prefer-primary/prefer-rr` results are the most easily misread from documentation alone. “Prefer” here does not mean a soft weighting across all nodes with a lean toward one type – measured directly, it means “use only the preferred set while any node in it remains available, and fall back to the other set only if it’s entirely unavailable.” Confirming this against real connection counts, rather than assuming from the setting’s name, avoided a real misunderstanding about how evenly load actually spreads under `prefer-*`.

7.1.3 Related Settings Worth Knowing

Beyond `load_balance`, several other settings shape how the driver actually behaves, each confirmed against the driver’s own documentation and this cluster’s configuration:

- **topology_keys** (or `topology-keys`, spelling depends on the specific driver/language binding) – restricts connections to nodes matching specific `cloud.region.zone` patterns, for geo-aware routing (e.g., keeping application traffic within one region under normal conditions).
- **fallback_to_topology_keys_only** (default `false`) – when `true`, the driver will *only* ever connect to nodes matching `topology_keys`, even if none are available (failing rather than falling back); when `false` (the default), it falls back to any available node if none match the preferred topology.
- **yb_servers_refresh_interval** (default 300 seconds) – how often the driver re-queries cluster topology. This is the number that governs how quickly the driver notices a node has been added, removed, or blacklisted – not instantly, but on this polling cadence.
- **SET yb_read_from_followers = true;** (session-level) – allows read queries to be served from tablet followers rather than requiring the tablet leader, trading strict read-your-writes consistency within that session for the ability to spread read load beyond just leaders.

[!NOTE] Not every yugabyteDB driver/language binding supports every one of these settings identically – confirm the specific setting name and default against the driver you’re actually

using (JDBC, psycopg, Node.js, etc.) rather than assuming full parity across all of them.

7.1.4 Blacklisting: Two Different Mechanisms, Two Different Driver Reactions

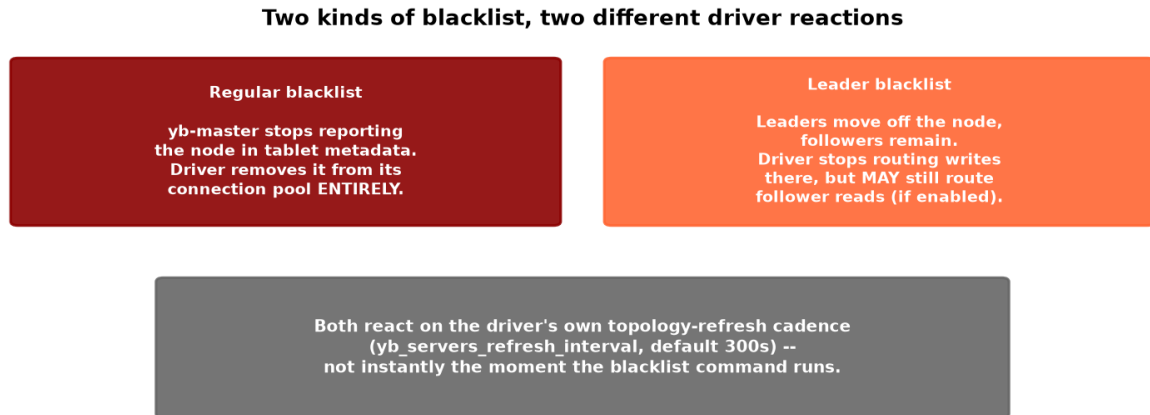


Figure 3: Figure 2: Two kinds of blacklist, two different driver reactions

yugabyteDB operators have two distinct blacklisting mechanisms, and the smart driver reacts to each differently – worth knowing precisely before relying on either during planned maintenance.

A regular (full) blacklist (`yb-admin ... change_blacklist ADD <tserver-uuid>`) removes a node from serving *any* request, read or write – yb-master stops reporting it at all in tablet location metadata, tablet leaders and followers both move off it, and it becomes effectively invisible to the cluster. The driver's reaction: on its next topology refresh (governed by `yb_servers_refresh_interval`), the node disappears from the driver's own connection pool entirely, and all subsequent connections/queries route elsewhere.

A leader blacklist (`yb-admin ... change_leader_blacklist ADD <tserver-uuid>`) is more targeted – it prevents the node from hosting tablet *leaders*, but it can still host followers and serve follower reads. Typical uses are preparing for a rolling restart (reducing leader churn ahead of time) or degrading a node's write/consistent-read role while keeping it usable for reads. The driver's reaction here is correspondingly narrower: writes and leader-requiring consistent reads stop routing to that node, but the driver **may still route follower reads there** if the session has `yb_read_from_followers = true` set – the node isn't removed from the pool outright, only demoted in role.

[!NOTE] Both reactions happen on the driver's own topology-refresh cadence, not the instant the blacklist command is issued at the cluster level – there is a real window (up to `yb_servers_refresh_interval`, 300 seconds by default) where the driver may still be routing based on slightly stale topology information. Lowering this interval ahead of a planned maintenance window is a reasonable, deliberate tradeoff between faster reaction and more frequent topology-discovery overhead.

7.2 Complete the following

In this section, we reproduce section 7.1.2's measured connection-distribution table directly, then reproduce the leader-blacklist behavior from section 7.1.4.

7.2.1 Prerequisites

A yugabyteDB cluster with at least one primary and one read-replica node (this project's real test used 3 of each), and a smart-driver-based client capable of reporting which physical node each acquired connection actually landed on.

7.2.2 Step 1 – Confirm your cluster's real topology

```
SELECT * FROM yb_servers();
```

Note which nodes are `primary` and which are `read_replica` – this is the ground truth the rest of this section's results should be checked against.

7.2.3 Step 2 – Reproduce the six-setting connection-distribution test

Acquire the same number of connections (6 is a convenient number if your cluster has 6 nodes, adjust proportionally otherwise) once per `load_balance` setting (`false`, `true`, `only-primary`, `only-rr`, `prefer-primary`, `prefer-rr`), reporting the physical host:port each connection actually lands on each time. Confirm your own results match the *pattern* in section 7.1.2's table (a single node for `false`, full spread for `true`, exclusive-set behavior for the other four) even if your own node counts differ from this project's 3-and-3 topology.

7.2.4 Step 3 – Reproduce the leader-blacklist behavior

```
yb-admin -master_addresses <masters> change_leader_blacklist ADD <tserver-uuid>
```

With a client connected using `load_balance=true` and `yb_read_from_followers=true`, confirm writes stop routing to the leader-blacklisted node while follower reads may still land on it – the asymmetric reaction described in section 7.1.4. Then remove the blacklist entry and confirm normal routing resumes on the driver's next topology refresh.

Section summary In this section, we reproduced the real, measured connection-distribution behavior behind each `load_balance` setting, and confirmed directly – rather than assumed from documentation – that a leader blacklist produces a narrower, role-specific change in driver routing than a full blacklist does.

7.3 In this document, we reviewed or created

This month and in this document we detailed the following:

- How the smart driver actually discovers cluster topology – the supplied `host/port` is only a starting contact point, not the final connection set, and should span multiple zones for resilience.
- Six real, measured connection-distribution outcomes, one per `load_balance` setting, against a live 6-node (3 primary, 3 read-replica) cluster – including the easily-misread `prefer-*` behavior (an exclusive-set preference, not a soft weighting) and the important fact that the default (`false`) setting performs zero distribution at all.
- Several related settings worth knowing (`topology_keys`, `fallback_to_topology_keys_only`, `yb_servers_refresh_interval`, session-level `yb_read_from_followers`), and the caveat that driver/language bindings don't necessarily support every setting identically.
- The real, differing driver reaction to a full (regular) blacklist versus a leader blacklist – complete removal from the connection pool versus a narrower loss of write/leader-read eligibility while follower reads may still be routed there – both bounded by the driver's own topology-refresh interval rather than taking effect instantly.

Persons who helped this month Kiyu Gabriel, David Bechberger

Additional resources: Free yugabyteDB training courses,

https://university.yugabyte.com/users/sign_in

Take any class, anytime, for free.

Jim Knicely's very excellent blog site,

<https://yugabytedb.tips/>

This document is located here,

<https://www.yugabitten.com/62%20-%20Monthly%20articles/2026-05%20-%20Cluster-Aware%20Smart%20Driver/05%20Cluster-aware%20smart%20driver.pdf>